

TrustNotify: A Lightweight Framework for Complete and Trustworthy Data Deletion Notification Distribution

Qipeng Song, Ruiyun Wang, Yue Li*, Yiheng Yan, Xingyue Zhu, Hui Li

School of Cyber Engineering, Xidian University, Xi'an 710126, China

{qpsong, liyue}@xidian.edu.cn, {ruiyunwang, yihengyan, xingyuezhu}@stu.xidian.edu.cn, lihui@mail.xidian.edu.cn

Abstract—In recent years, the right to deletion has been recognized by many privacy regulations, stipulating that data controllers not only delete data upon request but also inform other relevant controllers to remove the same data. Designing an efficient process for distributing and acknowledging deletion notifications across all controllers involved is challenging. These challenges include: 1) ensuring all controllers holding the data are notified, 2) securing notifications in untrusted networks against malicious threats, and 3) reducing compliance costs while handling a high volume of deletion requests. In this paper, we introduce TrustNotify, a comprehensive solution to these challenges. TrustNotify utilizes a cross-plane architecture, linking a regulatory plane and a service plane. The regulatory plane ensures notification completeness through a cuckoo filter-based data tracking algorithm, while the service plane implements a secure and efficient protocol for distributing notifications. This protocol integrates HMAC-based hop-by-hop verification, recursive acknowledgment, and probabilistic sampling, ensuring security and scalability. We implement a prototype of TrustNotify, and experiments show it is practical with good performance.

Index Terms—Right to deletion, deletion notification distribution and acknowledgment, privacy protection.

I. INTRODUCTION

In recent years, many laws and regulations (e.g., GDPR [1]) have mandated that data controllers (e.g., companies) protect individuals' "right to be forgotten." This right requires that, upon receiving a deletion request, data controllers must not only erase the data but also inform other controllers to do the same [2].

However, the technical implementation of the right to deletion lags behind legislative progress. Legally, data controllers must promptly delete personal data upon request, given certain conditions. Yet, the lack of effective technical procedures exposes them to significant compliance risks.

Technically, guaranteeing the right to deletion involves two key problems: 1) verifiable deletion within a single data controller: how to permanently delete the target data [3] and prove it has been irrecoverably removed [4]; 2) secure distribution of deletion notifications¹ among data controllers: ensuring every controller is correctly notified before deletion. While the first problem has received significant attention, the second, equally important, is addressed for the first time in this paper.

In the digital age, personal data flows across multiple controllers. Enforcing the right-to-be-forgotten requires notifying

all relevant controllers about the data to be deleted. Without secure distribution, notifications may be missed, leading to incorrect deletions (see Sec. III-B). Thus, *securely distributing deletion notifications is crucial to safeguarding the right to erasure, and it is the focus of this paper.*

Designing a secure deletion notification mechanism is challenging. First, efficiently tracking data circulation among controllers is crucial to ensure thorough erasure and transparency. Individually documenting circulation by each controller would significantly increase compliance costs. Second, as awareness of the right-to-be-forgotten grows—driven by media and privacy advocates—deletion requests will surge. Since compliance lacks economic benefit, only a lightweight, secure mechanism can be widely adopted to safeguard the right to deletion.

To address these challenges, this paper proposes a cross-layer solution by dividing the deletion right protection infrastructure into regulatory and service planes. The regulatory plane records data circulation efficiently, allowing low-overhead reconstruction to ensure accurate deletion notification distribution. The service plane implements a lightweight, secure protocol supported by a trusted execution environment (TEE).

This paper models deletion notification distribution as a spanning tree rooted at the data controller receiving the request, formalizing it as a bidirectional traversal verification problem. The regulatory plane uses cuckoo filters to record data circulation, supporting insertion, deletion, and tree reconstruction. On the service plane, message authentication codes, trusted execution environments, and probabilistic sampling enable low-overhead verification of notification propagation and acknowledgment, countering attacks in untrustworthy networks.

In summary, our main contributions are:

- 1) *Formulating Secure Deletion Notification Distribution.* We identify the importance of secure deletion notification distribution over untrusted networks and model it as a spanning-tree bidirectional traversal verification problem, aligned with the data circulation history.
- 2) *Cross-layer Cooperation Network Architecture.* We propose a cross-layer centralized architecture with a regulatory plane and a service plane. The regulatory plane records data circulation, while the service plane ensures compliance with the right-to-be-forgotten. This architec-

¹In this work, the message between user and data controller is called deletion request, between data controllers called deletion notification

ture supports collaborative solutions for secure deletion notification distribution.

- 3) *Cuckoo Filters Based Data Circulation Recording at Regulatory Plane.* We propose a privacy data flow storage mechanism using a cuckoo filter to ensure complete and trusted deletion notifications. It supports efficient insertion, precise queries, and accurate deletion, adapting to data growth for scalability and stability.
- 4) *Lightweight Secure Deletion Notification Protocol.* We design TrustNotify, a service protocol enabled by trusted execution environments like Intel SGX. It employs message authentication codes, hop-by-hop verification, recursive acknowledgments, and probabilistic sampling to securely distribute deletion notifications, providing lightweight and robust protection against attacks.

II. RELATED WORK

In recent years, as awareness of the right to deletion has grown, research in this area has gained significant attention. Existing studies can be broadly classified into the following categories.

Irrecoverable Data Deletion Researchers have explored various methods for permanently erasing data from physical media. Mo et al. [5] propose a secure cloud data deletion scheme using key modulation to avoid third-party reliance. Xiong et al. [6] design a secure self-destruction scheme for cloud data based on KP-TSABE, ensuring data self-destruction after a user-defined expiration. Li et al. [7] propose a file-level secure deletion method for Android, independent of the underlying file system. Xiong et al. [8] present a KDE-based secure deletion scheme for IoT devices. Gnatyuk et al. [9] review global data deletion standards and propose STM Shredder, a randomized overwriting method. RedFlash [10] ensures irrecoverable deletion on Flash memory by removing data and duplicates at a small performance cost.

Verifiable Data Deletion This research area focuses on validating that data has been conclusively erased, using methods like encryption key deletion, overwriting techniques, and blockchain technology. Encryption key deletion removes keys, making encrypted data inaccessible. Zheng et al. [11] and Ma et al. [12] introduce schemes enhancing fine-grained deletion verification. Tian et al. [13] use Merkle hash trees for verifiable proof of deletion. In overwriting-based verification, random data blocks overwrite data, with servers generating verification values. Bauer et al. [14] implement secure deletion in the ext2 file system, while Bhat et al. [15] improve efficiency with block-level overwriting. Du et al. [16] and Yang et al. [17] propose fine-grained deletion and public verification schemes using advanced cryptographic techniques. Blockchain technology provides verifiable data deletion without third-party reliance.

Machine Unlearning Machine unlearning aims to remove data points from a machine learning model's training set after training [18]. SISA [19] partitions data to minimize the impact of a single data point, making unlearning more efficient.

Sekharia et al. [20] focus on exact forgetting with a computationally efficient unlearning algorithm. BAERASER [21] defends against backdoor attacks by reversing the injection process through machine unlearning. Verifying the effectiveness of machine unlearning has also been studied. Sommer et al. [22] propose a backdoor-based verification approach using hypothesis testing, while Eisenhofer et al. [23] introduce a cryptographic framework with SNARKs and Merkle trees for verifiable unlearning. However, machine unlearning can unintentionally leave data traces, posing privacy risks.

Discussion Our literature review shows that existing studies on the right to deletion mainly focus on irrecoverable and verifiable data erasure. To our knowledge, no research has addressed cross-domain deletion notification distribution before data deletion. This paper is the first to tackle this issue.

III. PROBLEM STATEMENT

A. How does Deletion Notification Distribution Work?

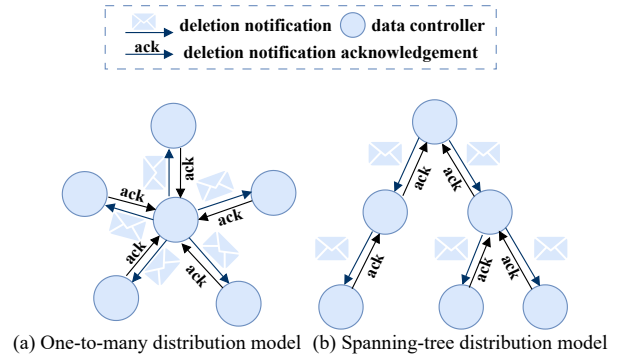


Fig. 2: Distribution model of deletion notifications among data controllers

Although the right to erasure mandates that all data controllers sharing the target data must execute reasonable deletion actions, current laws and regulations do not specify how involved controllers should be notified. Neither industry nor academia has addressed this procedural gap. Intuitively, one might assume a straightforward “one-to-many” model, as shown in Fig. 2(a), where the first data controller (the “source controller”) receiving the deletion request notifies all other controllers holding the data.

However, we argue that this approach imposes an unreasonable burden on the source data controller. For example, if data controller A shares data with B, who then shares it with 1,000 other controllers, A would be required to notify all 1,001 controllers to delete the data upon a user’s request. This is clearly unfair to A, as it bears most of the compliance costs while other controllers benefit from the shared data with minimal obligations. Such a model is unlikely to be acceptable to data controllers.

In light of this, it is more reasonable to assume that the distribution model of deletion notifications among data controllers follows the model illustrated in Fig. 2(b). In this model, the source data controller acts as the root node, distributing notifications to all successor controllers until the final leaf nodes are notified. The paths of deletion notifications

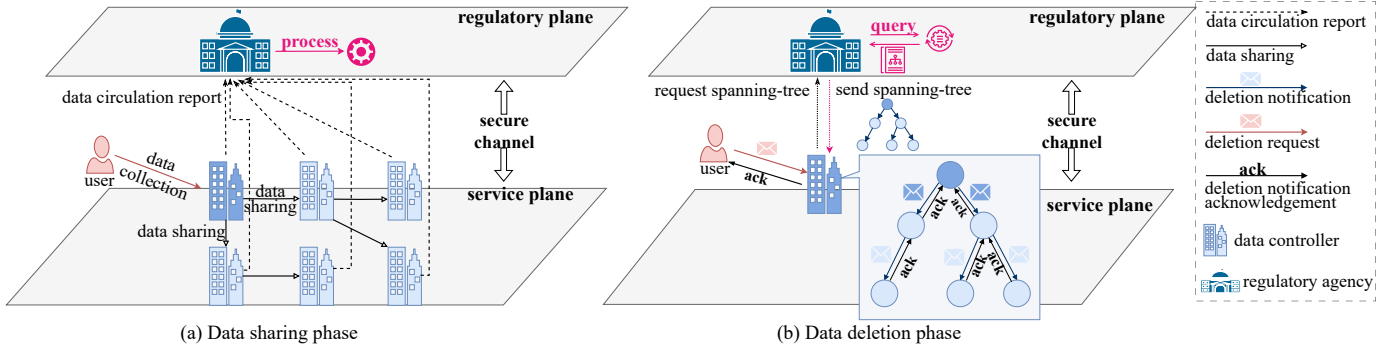


Fig. 1: Overview of the workflow for TrustNotify

form a spanning tree with the source controller at the root, encompassing all relevant controllers and aligning with the historical circulation path of the data. This model ensures that each controller reasonably participates in the distribution process. Compared to the one-to-many model, it reduces the computational and network resource burden on the source node and embodies the legal principle of “He who benefits, bears the responsibility.”

B. Threat Model

Data controllers are strongly incentivized to protect clients’ right to deletion due to potential financial penalties, legal action, and the rise of technologies enabling users to verify data deletion (e.g., machine unlearning [19], verifiable data deletion [3]).

In cross-data-controller scenarios, enforcing the right to deletion involves many controllers across regions. Distribution of deletion notifications over untrusted networks is vulnerable to disruption and security threats:

Tampering of Deletion Notification: Tampering involves unauthorized modification of a deletion notification, altering the user’s intent. For example, Alice requests data deletion, but an attacker intercepts and changes the request to delete Bob’s data instead.

Impersonation: Impersonation occurs when Eve, posing as a legitimate controller, tricks the system into executing a malicious deletion request. For instance, after obtaining Alice’s ID, Eve sends a fake deletion notification, leading to the unauthorized deletion of Alice’s data.

Path Deviation: Path deviation occurs when Eve redirects a deletion notification to an unrelated controller by altering the address. For example, Eve intercepts a notification meant for N and reroutes it to P. As a result, P deletes Alice’s data, while N fails to act, disrupting Alice’s services.

Replay Attack: A replay attack occurs when Eve captures and later resends a legitimate deletion notification, tricking the controller into accepting it as valid. For instance, Eve intercepts a notification for Alice’s data and resends it after Alice uploads new data, causing the controller to mistakenly delete the new data.

Drop Attack: In a drop attack, Eve intercepts and discards a deletion notification, then sends a forged acknowledgment, tricking the previous controller into believing the deletion was

successful. For instance, Eve discards a notification meant for N and falsely informs M of its completion, leaving Alice’s data undeleted.

We assume the attacker cannot observe or modify any data or code within an enclave, which is trusted to attest to its integrity. Although existing solutions like SGX, on which we build, don’t fully meet this assumption, addressing their security issues is beyond this work’s scope and remains an active research area [24], [25].

C. Design goals

1) *Completeness:* Completeness ensures that all intended data controllers receive deletion notifications, which is essential for safeguarding the right to deletion.

2) *Security:* Deletion notifications distributed over untrusted networks are vulnerable to various security threats. Our goal is to ensure the integrity and security of this process, protecting it from attacks outlined in the Threat Model.

3) *Lightweight:* We aim to build a lightweight, low-overhead mechanism, enabling data controllers to comply with the right to deletion, despite the lack of direct economic benefit.

IV. DESIGN

We propose a centralized management architecture with cross-layer collaboration to ensure secure distribution of deletion notifications among data controllers (see Fig. 1). The architecture includes regulatory and service planes. The regulatory plane, managed by a central agency, records data circulation, while the service plane has controllers report circulation data before sharing. Upon receiving a deletion request, the controller queries the regulatory plane, constructs the notification, and initiates distribution, then receives acknowledgment confirming all relevant controllers are notified.

Why prefer a centralized architecture over a distributed one? Centralized management enhances transparency, trust, and regulatory compliance. A trusted regulatory agency offers objective recording and verification, aiding in dispute resolution and legal audits. Conversely, distributed solutions increase compliance costs and require robust internal controls for accurate records and regulatory support. Moreover, some privacy laws mandate prior regulatory approval for sharing sensitive data. Thus, we propose a cross-layer centralized

architecture where regulatory and service planes coordinate to protect the right to deletion.

A. Regulatory Plane

The regulatory plane, managed by the agency, records data-sharing history among controllers. We implement the trusted deletion notification mechanism as a Network Function (NF) within a Trusted Execution Environment (TEE) like Intel SGX, with enclaves on each controller's server. Controllers are assumed to be trustworthy, and all notification modules use the same symmetric key, reducing compliance costs. How to periodically update this symmetric key is not considered in this paper.

Symbol	Meaning
D_{ID}	ID of the data pending deletion
U_{ID}	ID of the user to whom the data pending deletion belongs
C_{ID}^{first}	ID of the data controller who first collected the data
C_{ID}^{last}	the data controller ID of the current data source
C_{ID}^{des}	the next data controller to receive the deletion notification
$baseFp$	base fingerprint
$compositeFp$	composite fingerprint
$bucketSize$	the number of fingerprints that can be stored in a bucket
$bucketPow$	use it to calculate the size of the bucket array
i_1, i_2	two positions of the composite fingerprint in bucket

TABLE I: Symbol explanation

A key challenge in centralized management is the risk of single-point failure due to the large volume of data circulation history, requiring an efficient and scalable recording mechanism. A simple solution is storing circulation data in plaintext lists, like SQL databases, but this approach lacks storage efficiency and struggles with fast querying for large datasets. To ensure reliable and complete deletion notifications, we propose storing encoded fingerprints, reducing storage burden and improving processing efficiency. Only four attributes are required: data ID, user ID, source controller ID, and previous hop controller ID (see Tab. I).

The Cuckoo filter is efficient in data storage and querying due to its low memory usage and fast query speed, but it cannot map data relationships. To address this, we introduce base and composite fingerprints, deploying filters for each data controller at the regulatory level. This enables efficient insertion, querying, and deletion of circulation history, effectively managing privacy data flow among controllers.

Next, we introduce the circulation history insertion, query, deletion operations, and the dynamic expansion mechanism (see Alg. 1).

Circulation History Insertion Operation: When private data flows between data controllers, the receiving party inserts the circulation history into the corresponding filter in the regulatory plane (see Fig. 3(a)). The steps are as follows:

- 1) *Generation of the base fingerprint* First, a base fingerprint is generated using the MetroHash algorithm based on the triplet $(U_{ID}, D_{ID}, C_{ID}^{first})$. MetroHash is an efficient algorithm that processes large data volumes quickly with excellent random distribution and a very low probability

of hash collisions. The formula for the base fingerprint is as follows:

$$baseFp = \text{MetroHash}(U_{ID} \| D_{ID} \| C_{ID}^{first}) \quad (1)$$

The base fingerprint is later used to calculate the storage location of the composite fingerprint.

Algorithm 1 Operations details

```

procedure INSERT( $U_{ID}, D_{ID}, C_{ID}^{first}, C_{ID}^{last}$ )
   $i_1, compositeFp \leftarrow getIndexAndFp$ 
  if  $cf.insert(compositeFp, i_1)$  then return true
  end if
   $baseFp \leftarrow GenBaseFp$ 
   $i_2 \leftarrow getAltIndex(baseFp, i_1, cf.bucketPow)$ 
  if  $cf.insert(compositeFp, i_2)$  then return true
  end if return  $cf.reinsert(baseFp, compositeFp, randi(i_1, i_2))$ 
end procedure
procedure LOOKUPCIRCULATIONTREE( $U_{ID}, D_{ID}, C_{ID}^{first}$ )
   $baseFp \leftarrow GenBaseFp(U_{ID}, D_{ID}, C_{ID}^{first}); edges \leftarrow \emptyset$ 
  for each  $cf$  in AllDataControllersFilters do
     $i_1 \leftarrow uint(baseFp \gg 32) \& masks[cf.bucketPow]$ 
     $i_2 \leftarrow getAltIndex(baseFp, i_1, cf.bucketPow)$ 
    for each  $i$  in  $[i_1, i_2]$  do
      for each  $fp$  in  $cf.buckets[i]$  do
        if  $HasPrefix(fp, fmt("%x", baseFp))$  then
           $C_{ID}^{last} \leftarrow fp[len(fmt("%x", baseFp)) :]$ 
          Add edge from  $C_{ID}^{last}$  to  $cf.ID$  in  $edges$ 
        end if
      end for
    end for
  end for
  if  $edges \neq \emptyset$  then return Circulation tree from  $edges$ 
  elsereturn false
end if
end procedure
procedure DELETECIRCULATIONTREE( $U_{ID}, D_{ID}, C_{ID}^{first}$ )
   $baseFp \leftarrow GenBaseFp(U_{ID}, D_{ID}, C_{ID}^{first}); DelCount \leftarrow 0$ 
  for each  $cf$  in AllDataControllersFilters do
     $i_1, compositeFp \leftarrow getIndexAndFp(U_{ID}, D_{ID}, C_{ID}^{first}, cf.ID)$ 
    if  $cf.delete(compositeFp, i_1)$  then
       $DelCount \leftarrow DelCount + 1$ 
    continue
    end if
     $i_2 \leftarrow getAltIndex(baseFp, i_1, cf.bucketPow)$ 
    if  $cf.delete(compositeFp, i_2)$  then
       $DelCount \leftarrow DelCount + 1$ 
    end if
  end for
  if  $DelCount > 0$  then return true
  elsereturn false
  end if
end procedure

```

- 2) *Generation of the composite fingerprint* Next, the base fingerprint is combined with C_{ID}^{last} to generate and store the composite fingerprint in the filter. The formula is:

$$compositeFp = baseFp \| C_{ID}^{last} \quad (2)$$

- 3) *Insertion of the composite fingerprint* The composite fingerprint has two potential positions (i_1, i_2) to handle hash collisions. First, the first position (i_1) is calculated, and an attempt is made to insert the fingerprint:

$$i_1 = \text{baseFp} \gg 32 \& (2^{\text{bucketPow}} - 1) \quad (3)$$

If the first insertion fails, a second location i_2 is calculated using the base fingerprint and i_1 , and another insertion attempt is made:

$$i_2 = (i_1 \oplus \text{hash}(\text{baseFp})) \& (2^{\text{bucketPow}} - 1) \quad (4)$$

If the composite fingerprint cannot be inserted at either location, a reinsertion operation is triggered. A position between i_1 and i_2 is randomly selected, an entry is swapped out, and the displaced fingerprint is inserted into its other potential location. This may trigger a chain reaction, with multiple fingerprints swapped until a free slot is found or the swap limit is reached.

Circulation History Query Operation: When a data controller receives a deletion notification, the circulation path is retrieved from the regulatory level, which conducts the Circulation History Query Operation as follows:

- 1) *Storage location calculation* The regulatory level first calculates the two potential storage locations (i_1, i_2) for the composite fingerprint using the base fingerprint derived from U_{ID} , D_{ID} , and C_{ID}^{first} .
- 2) *Search operation* Iterate through the buckets at i_1 and i_2 , examining each fingerprint. If a fingerprint's prefix matches the base fingerprint, decode the last component ID (C_{ID}^{last}) to identify the previous hop ID.
- 3) *Circulation history reconstruction* To reconstruct the circulation history, the regulatory agency asynchronously queries all filters except the specific data controller's (see Fig. 3(b)) to determine each controller's previous hop ID. By linking these IDs, directed edges are formed, allowing the agency to reconstruct the complete data circulation path.

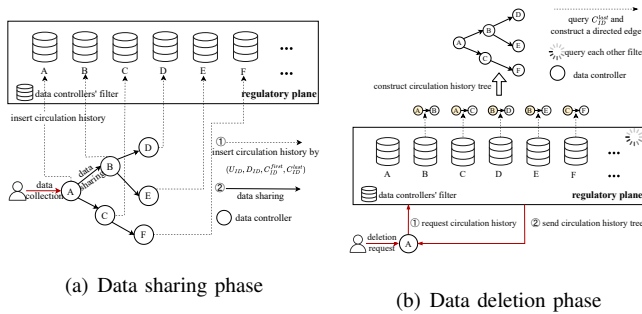


Fig. 3: Workflow of data circulation record insertion and query

Circulation History Deletion Operation: After a data controller deletes private data, the corresponding circulation path must be removed. The deletion operation, based on the query operation, starts by calculating the composite fingerprint and its two potential storage locations. These locations are traversed to find and compare the composite fingerprint. If a match is found, it is removed from the filter, accurately eliminating the circulation history.

Dynamic Expansion Mechanism: The dynamic expansion mechanism addresses the storage needs of large-scale circulation history records, allowing capacity to adjust with data growth. Specifically: when a filter's load factor (current capacity to total capacity ratio) reaches a preset threshold, the regulatory agency automatically increases storage capacity to accommodate more records (see in Alg. 2). This process is described by the following formula: $\text{NewCapacity} = \text{Currentsize} \times \text{bucketsize} \times 2$. When a filter reaches the predetermined load factor, the system automatically creates a new filter and adds it to the chain, dispersing storage pressure and maintaining efficiency.

Algorithm 2 Scale details

```

procedure SCALE(sf)
    needScale  $\leftarrow$  false
    lastFilter  $\leftarrow$  sf.filters[len(sf.filters) - 1]
    if (lastFilter.Count()  $\geq$  len(lastFilter.buckets) *
    sf.loadFactor) then
        needScale  $\leftarrow$  true
    end if
    if needScale then
        newCapacity  $\leftarrow$  scaleFactor(len(lastFilter.buckets))
        newFilter  $\leftarrow$  NewFilter(newCapacity)
        sf.filters  $\leftarrow$  append(sf.filters, newFilter)
    end if
end procedure

```

We record the source data controller ID instead of the destination ID to standardize composite fingerprint size and simplify data management. This prevents space waste and reduces query efficiency from multiple potential next hops. Since the next hop may change over time, recording the source avoids frequent updates. Asynchronous queries on all filters trade time complexity for space savings, enabling efficient data circulation tracking. As data grows, our solution dynamically expands storage, maintaining efficient processing and query performance.

B. Service Plane

Beyond completeness, deletion notifications in untrusted networks are vulnerable to tampering and replay attacks, threatening their reliability. The source data controller must verify the transmission path to ensure notifications follow the intended route. Thus, a protection mechanism at the service layer is essential for secure transmission and visibility of distribution results.

We divide the distribution and acknowledgment of deletion notifications into two subprocesses: Forward Traversal Verification and Recursive Backward Acknowledgment. Forward Traversal sends notifications step-by-step to leaf nodes with verification codes to prevent tampering; Recursive Backward confirms receipt, marking nodes as "red", "yellow", or "green" based on status. Finally, the initiating node ensures all nodes have received the notification.

1) *Forward Traversal Verification:* The TrustNotify protocol, designed as a hop-by-hop protocol, standardizes the handling and forwarding of deletion notifications, ensuring all

relevant data controllers receive them. The protocol header fields clearly identify and communicate the source, destination, and required actions, enabling precise control and oversight. The core header includes the user's unique ID (U_{ID}), notification type (Type), timestamp (T), first data controller ID (C_{ID}^{first}), target data controller ID (C_{ID}^{des}), operation request (Req), and authentication field (Auth).

The header fields of the protocol are designed as follows:

- U_{ID} : the unique identifier of the user in the deletion notification.
- Type: indicates the source of the deletion notification; Type = 0 for user-issued, Type = 1 for forwarded by the data controller, Type = 2 for acknowledgment.
- T: timestamp. The data controller calculates the time difference $\Delta t = t_2 - t_1$. C_{ID}^{first} is the previous data controller, irrelevant when Type = 0. C_{ID}^{des} is the next data controller, irrelevant when Type = 2.
- Req: identifies operations during the deletion process, like deleting all personal information, age, or identification numbers. Specific mappings and extended functions should be defined by the internet data controller based on their needs.
- Auth: generated using a quadruple and session key k , as follows:

$$\text{Auth} \leftarrow \text{HMAC}((U_{ID} \| T \| \text{Type} \| C_{ID}^{des}), \text{key}) \quad (5)$$

In the TrustNotify protocol, hop-by-hop verification (see Sec. IV-B1) ensures secure transmission of deletion notifications. When a notification is forwarded, the receiving data controller replaces the target ID (C_{ID}^{des}) in the header with its own ID and recalculates the authentication field Auth' .

$$\text{Auth}' \leftarrow \text{HMAC}((U_{ID} \| T \| \text{Type} \| C_{ID}^{own}), \text{key}) \quad (6)$$

Authentication at each hop is based on the recipient's unique information, enhancing transmission security. This hop-by-hop method enables the TrustNotify protocol to prevent tampering and forgery, ensuring secure and reliable transmission of deletion notifications among data controllers.

2) *Recursive Backward Acknowledgement*: We propose an innovative Traffic Light Mechanism (TLM) to manage the acknowledgment process of deletion notifications. TLM guides the transmission of acknowledgment messages through three phases: red, yellow, and green.

- Red Light Phase: Upon receiving a deletion notification, each data controller conducts a security check. If compromised, the controller reports to its superior, enters ALERT state, and halts the notification. If successful, it tracks pending acknowledgments from child nodes.
- Yellow Light Phase: After passing verification, the controller waits for acknowledgments from all subordinate nodes. Once received, it moves to the green light phase.
- Green Light Phase: The controller sends an acknowledgment to its superior, confirming all subordinate nodes have processed the notification and the branch is ready to resume normal operations.

3) *Probabilistic Sampling Verification*: To reduce communication costs during tag verification, we propose a probabilistic sampling method.

This mechanism dynamically adjusts the sampling probability based on the number of deletion notifications processed, reducing the operational burden. In practice, each data controller first applies a keyed pseudorandom function (PRF) to the Auth field using session key k , then compares the result with a predefined sampling probability: $\text{PRF}_k(\text{Auth}) \leq P_s$. If the conditions are met, the data controller verifies Auth and sends an acknowledgment. Each data controller maintains a counter, CountS, to track the number of processed deletion notifications. The sampling probability P_s is dynamically calculated based on CountS and a predefined constant u . As CountS increases, P_s decreases; when CountS is low, P_s increases, allowing more notifications to be verified. The calculation is as follows:

$$P_s(\text{CountS}) = \frac{1}{(1+u)^{\text{CountS}}} \quad (7)$$

Directly applying this sampling formula could lead to an excessively low sampling probability after processing many deletion notifications. To prevent this, we introduce a minimum sampling probability, $P_{\min}$, to ensure it doesn't fall too low. Data controllers should adjust parameters based on their scenarios to balance efficiency and cost. The final sampling probability formula is:

$$P_s(\text{CountS}) = \begin{cases} \frac{1}{(1+u)^{\text{CountS}}}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} > P_{\min} \\ P_{\min}, & \text{if } \frac{1}{(1+u)^{\text{CountS}}} \leq P_{\min} \end{cases} \quad (8)$$

Data controllers should adjust parameter configurations based on actual needs. For higher security, the sampling rate can be increased to enhance system performance. Controllers can also determine optimal settings based on factors like data volume, processing speed, and system load.

4) *Secure NF Execution*: To provide an out-of-the-box solution, we implement the protocol detailed in Sec. IV-B1 as a standalone network function. This function operates independently of data controllers' normal operations, focusing on distributing deletion notifications and informing systems responsible for executing data deletion.

Our trust distribution verification mechanism is implemented on Intel SGX. SGX, a leading Trusted Execution Environment (TEE) technology, creates a secure Enclave with extended instructions and memory access control, providing confidentiality and integrity for the code and data. In our scheme, command verification occurs within the Enclave, where verification keys are securely stored. The Enclave's isolation from the data controller system prevents attackers or dishonest controllers from accessing or tampering with the code, keys, or verification results, significantly enhancing the security and reliability of the verification mechanism.

V. SECURITY ANALYSIS

- 1) **Tampering of Deletion Notification** A malicious attacker, Eve, intercepts the deletion notification and alters Alice's ID to Bob's before sending it to the target data controller. The target data controller identifies the label mismatch by comparing the Auth tag.

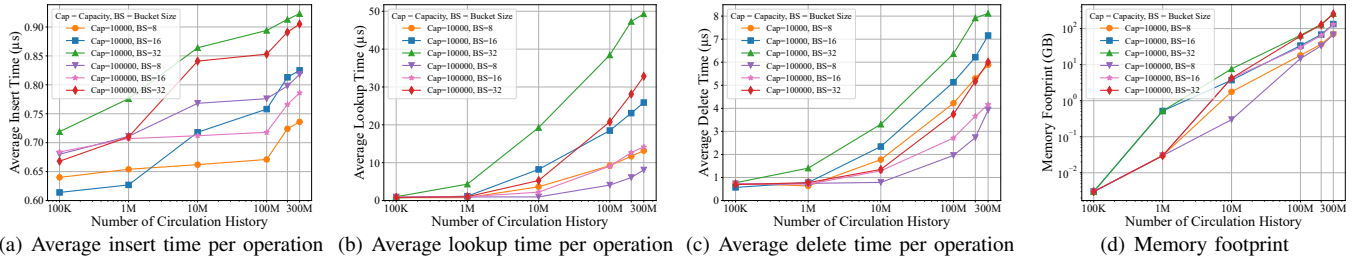


Fig. 4: Regulatory plane performance

- 2) **Impersonation** Having obtained Alice’s ID, the attacker Eve attempts to masquerade as a node in Alice’s information flow and sends a false deletion notification to the target data controller holding Alice’s private information. However, unaware of the session key k , Eve uses a different string in the label generation algorithm. The target data controller detects the mismatch through label verification.
- 3) **Path Deviation** A malicious attacker, Eve, intercepts Alice’s deletion notification, alters the destination, and sends it to a non-target data controller. The non-target controller detects the mismatch by comparing the Auth tag.
- 4) **Replay Attack** A malicious attacker, Eve, intercepts and saves a deletion notification without modification. After some time, it is resent to the target data controller, which identifies the replay attack by detecting the timestamp discrepancy Δt .
- 5) **Drop Attack** Data controller M forwards Alice’s deletion notification to N, but a malicious attacker, Eve, intercepts and discards it, then sends a false success acknowledgment to M. Upon verifying the response, M detects a mismatch, indicating an anomaly in the deletion process.

VI. PERFORMANCE EVALUATION

A. Implementation and Experimental Setup

We developed a prototype system for the proposed solution. On the regulatory plane, we used Go to implement a data circulation tree query mechanism based on cuckoo filters. On the service plane, we implemented the secure distribution protocol for deletion notifications using the Python Sanic framework², with asynchronous I/O and RESTful APIs.

Our experiments use containernet, a network emulation tool based on mininet with Docker integration³. In the emulated network, each Docker container acts as a data controller. To simulate the deletion notification distribution and evaluate our proposal’s performance in real-world networks, we employ four notable topologies—Chinanet, Karen, Lambdanet, and Viatel—from the Internet Topology Zoo [26]. These topologies are described in GraphML format, which contains diverse network nodes and connections. Containernet emulation starts by parsing GraphML into Docker containers and virtual links.

²<https://github.com/sanic-org/sanic>

³<https://github.com/containernet>

The emulation program is executed on a Virtual Machine (VM) hosted by Aliyun Cloud. The VM’s configuration for the study is as follows: an ecs.g7t.16xlarge cloud server, supporting Intel SGX for trusted execution environments, and equipped with 128 GB of encrypted memory. Additionally, the server is provisioned with 64 vCPUs, 256 GB of memory, and 32 GiB of network bandwidth.

B. Performance Evaluation

1) **Regulatory Plane:** We conducted experiments to evaluate the performance of our proposed solution for documenting data circulation records, as detailed in Sec. IV-A. Our evaluation focuses on two key metrics: average operation time and memory footprint, analyzed during insertion, querying, and deletion operations. The experimental setup considers only the default capacity and bucket size, which significantly impact storage and query efficiency. Due to testing limitations, we set default capacities at 10,000 and 100,000, and bucket sizes at 8, 16, and 32.

Experimental results are shown in Fig.4. The x-axis represents the number of circulation histories (log scale), while the y-axes in Figs.4(a), (b), and (c) display the average times for insertion, search, and deletion of circulation histories. Fig. 4(d) illustrates the memory footprint after varying numbers of circulation histories.

In Fig. 4(a), as circulation history records increase, insertion time slightly rises but stays below 1 microsecond, even with 300 million records, indicating efficient handling of large-scale data. Higher capacities with the same bucket size lead to smaller time increases due to fewer dynamic expansions. However, larger bucket sizes increase insertion time because, in cuckoo hashing, if both assigned positions are occupied, elements must be relocated, potentially triggering chain reactions that prolong resolution time.

Next, in Fig. 4(b), search time increases with more circulation histories, which is expected as more positions must be traversed. Higher capacities with the same bucket size result in shorter search times due to lower load factors and sparser element distribution. Finally, bucket sizes of 8 and 16 perform better, while a size of 32 leads to longer search times due to increased comparisons within the bucket.

The average deletion time per circulation history (see Fig. 4(c)) is similar to the search time, as deletion is based on the search operation.

Next, we analyze the memory footprint in Fig.4(d). As the number of circulation histories increases, the memory footprint

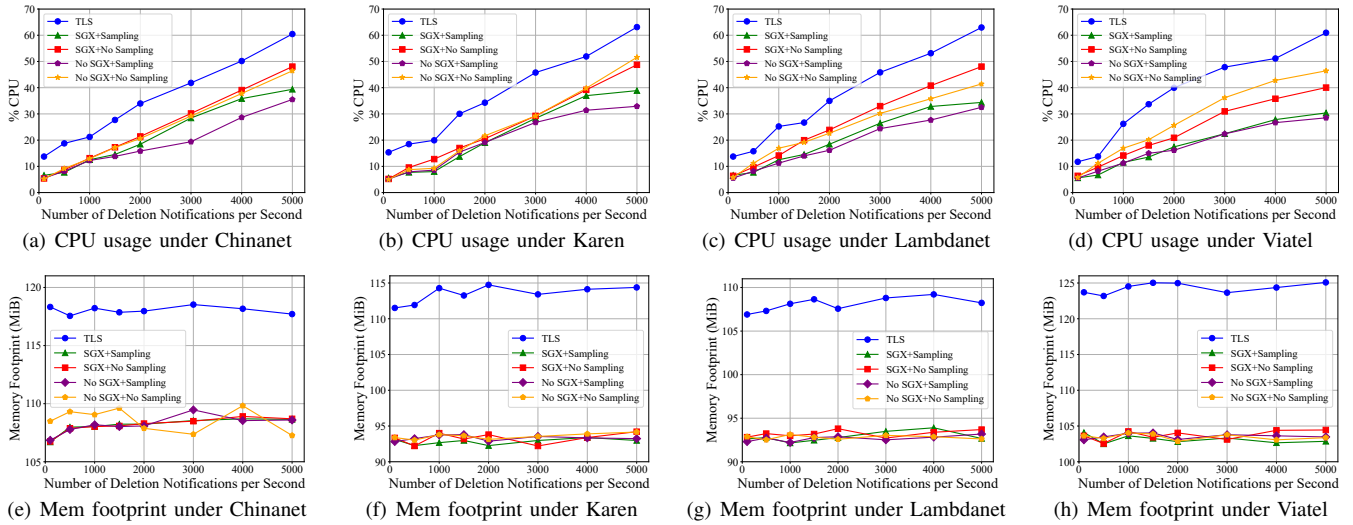


Fig. 5: CPU Usage and memory footprint with respect to the intensity level of deletion notifications under different network topology

grows, as expected, since more elements require more space. With the same bucket size, smaller default capacities result in a higher memory footprint due to more frequent dynamic expansions when capacity limits are reached (see sec.IV-A). Conversely, with the same default capacity, a smaller bucket size leads to a lower memory footprint because fewer histories per bucket reduce overall memory consumption.

As circulation history increases, the memory footprint becomes more similar across configurations with the same bucket size. This occurs because the expansion mechanism eventually creates the final filter and allocates significant memory, reducing the impact of initial default capacity. This pre-allocation strategy quickly accommodates growth in data circulation records, minimizing performance degradation from frequent expansions.

Our analysis concludes that the proposed centralized management architecture (see sec. IV) can handle large-scale data circulation history. Experimental results indicate that using a smaller bucket size and larger initial capacity effectively reduces time and memory costs. However, the regulatory agency should adjust filter parameters based on the number of circulation histories to optimize performance. While storing 300 million records may require about 200GB of memory, storage capacity can be improved through technical methods. For example, dividing regulatory agencies by region can alleviate pressure on a single agency, or storing records on hard drives, which sacrifices some query efficiency but significantly increases capacity.

2) **Service Plane:** To evaluate the system resource consumption of our NF on the service plane, we conduct experiments focusing on two key metrics: CPU usage and memory consumption. As a baseline, we use a TLS-based scheme where adjacent data controllers use TLS for secure deletion notification delivery—this is the most straightforward approach. We also assess the resource costs within and outside TEE to evaluate its security benefits. Additionally, we measure

the CPU and memory footprint of NF with and without adaptive probabilistic sampling. Finally, we test our proposal on four representative network topologies: Chinanet, Karen, Lambdanet, and Viatel.

Performance evaluation results are shown in Fig. 5. In each subfigure, the x-axis represents the number of deletion notifications per second received by each data controller, indicating the intensity of notifications, while the y-axis shows the corresponding CPU usage or memory footprint.

Using the Chinanet topology as an example, particularly in terms of CPU usage shown in Fig. 5(a), we observe that the TLS-based method exhibits the highest CPU consumption. CPU usage increases with the intensity of deletion notifications, reaching about 60% at 5000 notifications per second, significantly higher than other methods. This high demand is due to the TLS method’s reliance on computationally intensive asymmetric encryption and decryption. In contrast, TrustNotify uses symmetric cryptography, substantially lowering CPU usage. This reduction implies that, under our solution, a data controller can handle a higher volume of deletion notifications more efficiently.

The use of SGX leads to a minimal increase in CPU usage, mainly due to I/O operations between enclave and non-enclave areas. This slight rise in demand is because only essential functionalities, specifically the verification components, are designed within the SGX enclave, minimizing the computational burden inside.

Regarding CPU consumption with and without adaptive probabilistic sampling, a noticeable difference occurs when deletion notifications exceed 1000 per second. As notification intensity increases, the benefit of sampling becomes evident, significantly lowering CPU usage. This reduction occurs because sampling decreases the number of notifications the CPU must verify. Consequently, data controllers can adjust sampling settings to balance system performance with processing demands.

In Fig. 5(e), the TLS solution's memory footprint is around 118MB, while our solution uses about 107MB, approximately 10% less. This difference is due to TLS's reliance on asymmetric encryption, which requires more memory for key storage and intermediate computations. Notably, SGX does not significantly impact memory usage, as it is designed to enhance security without increasing memory footprint. Similarly, sampling does not substantially affect memory, as it primarily reduces CPU workload and processing times. Overall, memory footprint remains stable, driven by the I/O-intensive nature of operations, which are not heavily influenced by data volume. The rise in CPU usage with increasing data volume is linked to asynchronous or non-blocking I/O, which improves efficiency by allowing the CPU to handle other tasks during I/O operations, thus enhancing overall application performance.

The experimental results under the other three network topologies (see Fig. 5(b) to Fig. 5(h)) exhibit a similar trend to those under the Chinanet topology, revealing that our solution is independent of network topology. This is due to the hop-by-hop verification design (detailed in sec. IV-B1), which decouples our proposal from the network topology. As a result, our solution can handle any network topology and data circulation tree.

VII. CONCLUSION

In an era of widespread personal data circulation, ensuring the complete and secure distribution of deletion notifications among data controllers is crucial to safeguarding the right to deletion. We are the first to identify this challenge and have developed TrustNotify to address it. TrustNotify, built on a cross-plane collaborative centralized architecture, features a cuckoo filter-based data circulation recording algorithm for regulatory agencies and a secure, lightweight deletion notification distribution protocol for data controllers.

Our experimental results show that the cuckoo filter-based algorithm effectively handles extensive data circulation history and efficiently reconstructs the data circulation tree for enforcing the right to deletion. Performance evaluations confirm that the proposed protocol can manage a large volume of deletion notifications securely and efficiently. In future work, we aim to address potential privacy breaches arising from the distribution of deletion notifications, an aspect not yet covered in this study.

REFERENCES

- [1] A. Mantelero, "The eu proposal for a general data protection regulation and the roots of the 'right to be forgotten'," *Computer Law & Security Review*, vol. 29, no. 3, pp. 229–235, 2013.
- [2] Data Protection Commission, "The Right to Erasure (Articles 17 & 19 of the GDPR)," Available: <https://www.dataprotection.ie/en/individuals/know-your-rights/right-erasure-articles-17-19-gdpr>, 2022, accessed: Jan. 30, 2024.
- [3] J. Bartusek, D. Khurana, and A. Poremba, "Publicly-verifiable deletion via target-collapsing functions," in *Annual International Cryptology Conference*, 2023, pp. 99–128.
- [4] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4007–4022.
- [5] Z. Mo, Y. Qiao, and S. Chen, "Two-party fine-grained assured deletion of outsourced data in cloud systems," in *2014 IEEE 34th International Conference on Distributed Computing Systems*, 2014, pp. 308–317.
- [6] J. Xiong, X. Liu, Z. Yao, J. Ma, Q. Li, K. Geng, and P. S. Chen, "A secure data self-destructing scheme in cloud computing," *IEEE Transactions on Cloud Computing*, vol. 2, no. 4, pp. 448–458, 2014.
- [7] L. Yang, T. Wei, F. Zhang, and J. Ma, "Sadus: Secure data deletion in user space for mobile devices," *computers & security*, vol. 77, pp. 612–626, 2018.
- [8] J. Xiong, L. Chen, M. Z. A. Bhuiyan, C. Cao, M. Wang, E. Luo, and X. Liu, "A secure data deletion scheme for iot devices through key derivation encryption and data analysis," *Future Generation Computer Systems*, vol. 111, pp. 741–753, 2020.
- [9] S. Gnatyuk, V. Kinzeryavyy, T. Sapozhnik, I. Sopilko, N. Seilova, and A. Hrytsak, "Modern method and software tool for guaranteed data deletion in advanced big data systems," in *Advances in Artificial Systems for Medicine and Education II 2*, 2020, pp. 581–590.
- [10] N. Chen and B. Chen, "Duplicates also matter! towards secure deletion on flash-based storage media by removing duplicates," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, 2022, pp. 54–66.
- [11] D. Zheng, L. Xue, C. Yu, Y. Li, and Y. Yu, "Toward assured data deletion in cloud storage," *IEEE Network*, vol. 34, no. 3, pp. 101–107, 2020.
- [12] J. Ma, M. Wang, J. Xiong, and Y. Hu, "Cp-abe-based secure and verifiable data deletion in cloud," *Security and Communication Networks*, vol. 2021, pp. 1–14, 2021.
- [13] J. Tian and Z. Wang, "Cloud data assured deletion scheme based on dynamic sliding window," *Peer-to-Peer Networking and Applications*, vol. 15, no. 4, pp. 1817–1833, 2022.
- [14] S. Bauer and N. B. Priyantha, "Secure data deletion for linux file systems," in *10th USENIX Security Symposium (USENIX Security 01)*, 2001.
- [15] W. Bhat and S. Quadri, "Restfs: Secure data deletion using reliable & efficient stackable file system," in *2012 IEEE 10th International Symposium on Applied Machine Intelligence and Informatics (SAMII)*, 2012, pp. 457–462.
- [16] X. H. Ruizhong DU, Pengliang SHI, "Cloud data assured deletion scheme based on overwrite verification," *Journal on Communications*, vol. 40, no. 1, pp. 130–140, 2019.
- [17] C. Yang, Y. Liu, X. Tao, and F. Zhao, "Publicly verifiable and efficient fine-grained data deletion scheme in cloud computing," *IEEE Access*, vol. 8, pp. 99 393–99 403, 2020.
- [18] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and Y. Zhang, "When machine unlearning jeopardizes privacy," in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 896–911.
- [19] L. Bourtoutle, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 141–159.
- [20] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, "Remember what you want to forget: Algorithms for machine unlearning," *Advances in Neural Information Processing Systems*, vol. 34, pp. 18 075–18 086, 2021.
- [21] Y. Liu, M. Fan, C. Chen, X. Liu, Z. Ma, L. Wang, and J. Ma, "Backdoor defense with machine unlearning," in *IEEE INFOCOM 2022-IEEE conference on computer communications*, 2022, pp. 280–289.
- [22] D. M. Sommer, L. Song, S. Wagh, and P. Mittal, "Towards probabilistic verification of machine unlearning," *arXiv preprint arXiv:2003.04247*, 2020.
- [23] T. Eisenhofer, D. Riepel, V. Chandrasekaran, E. Ghosh, O. Ohrimenko, and N. Papernot, "Verifiable and provably secure machine unlearning," *arXiv preprint arXiv:2210.09126*, 2022.
- [24] S. Constable, J. Van Bulck, X. Cheng, Y. Xiao, C. Xing, I. Alexandrovich, T. Kim, F. Piessens, M. Vij, and M. Silberstein, "Aex-notify: Thwarting precise single-stepping attacks through interrupt awareness for intel sgx enclaves," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4051–4068.
- [25] A. Kogler, D. Gruss, and M. Schwarz, "Minefield: A software-only protection for sgx enclaves against dvfs attacks," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4147–4164.
- [26] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, 2011.